🎓 Cours & Antisèche : Pydantic, SQLAlchemy & SQLModel

Synthèse Universitaire des Notebooks 007, 008 & 009 + Recettes du Brief CF-Tech

🛡️ 007. PYDANTIC (Le Vigile)

Rôle : Valider et nettoyer les données en mémoire vive *avant* la BDD. Il ignore tout de SQL.

- **Coercition :** Convertit `"19"` en `19` ou lève `ValidationError`.
- `Field(ge=0, max_length=50)`
- `@field_validator` : Règle sur 1 champ (ex: âge ≥ 16 ans).
- `@model_validator` : Compare 2 champs (`date_debut < date_fin`).

⚙️ 008. SQLALCHEMY (L'Architecte)

Rôle : Traduire les classes Python en tables SQL et exécuter les requêtes relationnelles (ORM).

- **Engine :** Tuyau de connexion unique vers la BDD (`create_engine`).
- **Session :** Panier transactionnel (`add`, `commit`, `rollback`).
- **Style 2.0 :** `select(Table).where(...)` + `session.exec()`.

✨ 009. SQLMODEL (Le Mariage)

Rôle : Une seule classe hérite à la fois de Pydantic et de SQLAlchemy.

- `table=False` : Se comporte comme un schéma Pydantic pur (valide les entrées).
- `table=True` : Devient une vraie table SQL.
- Permet la **famille de modèles par héritage** (Base → Create → Table).

1. 📥 Entrée Brute
Formulaire Streamlit ou ligne de `legacy_erp_cftech.db`

→

2. 🛡️ Schéma Pydantic (table=False)
`ApprenantCreate.model_validate()`
Vérifie âge ≥ 16, EmailStr, Regex

Si Valide →

3. 🧱 Modèle SQLModel (table=True)
`Apprenant.model_validate()` + `session.add() & commit()`

/ Sinon →

❌ **ValidationError**
Intercepté par `try/except`
Rejeté avant la BDD !

1. PYDANTIC : VALIDER 1 CHAMP VS 2 CHAMPS

Dans `app/schemas/`, utilisez ces deux décorateurs pour bloquer les mauvaises saisies dès la porte d'entrée :

```
from datetime import date, datetime
import re
from pydantic import BaseModel, EmailStr, Field, field_validator, model_validator

class ApprenantCreate(BaseModel):
    nom: str = Field(min_length=1, max_length=50)
    prenom: str = Field(min_length=1, max_length=50)
    email: EmailStr
    date_naissance: date
    telephone: str | None = None

    # 1. Valider UN champ (@field_validator)
    @field_validator("date_naissance")
    @classmethod
    def verifier_age_16_ans(cls, v: date) → date:
        today = date.today()
        age = today.year - v.year - ((today.month, today.day) < (v.month, v.day))
        if age < 16 or v > today:
            raise ValueError(f"Âge invalide ({age} ans) : minimum 16 ans requis.")
        return v

    @field_validator("telephone")
    @classmethod
    def verifier_telephone(cls, v: str | None) → str | None:
        if v is not None and not re.match(r"^0[1-9]\d{8}$", v):
            raise ValueError("Téléphone invalide (10 chiffres, ex: 0612345678).")
        return v

class SessionCreate(BaseModel):
    titre: str = Field(max_length=100)
    date_debut: datetime
    date_fin: datetime
    salle_id: int
    formateur_id: int
    capacite_max: int = Field(ge=1)

    # 2. Comparer DEUX champs entre eux (@model_validator)
    @model_validator(mode="after")
    def verifier_coherence_dates(self):
        if self.date_fin ⩽ self.date_debut:
            raise ValueError("La date_fin doit être strictement après date_debut
!")
        return self
```

2. LE PIÈGE MAJEUR DU COURS 009 (SQLMODEL)

⚠️ **RÈGLE D'OR (Section 5 du Notebook 009) :**
Une classe avec `table=True` **NE VALIDE PAS** les données à sa création ! Si vous faites `Apprenant(nom="", email="faux")` directement sur la table, SQLModel laissera passer sans erreur Pydantic.

La Solution : Convertir le Schéma en Modèle

```
# ❌ MAUVAIS : Instancier la table directement (aucune validation !)
eleve = Apprenant(nom="", email="pas_un_email", date_naissance=date(2020, 1, 1))

# ✅ BON : Passer par ApprenantCreate PUIS convertir en Apprenant
donnees_valides = ApprenantCreate.model_validate(dictionnaire_brut)
eleve_db = Apprenant.model_validate(donnees_valides)

with Session(engine) as session:
    session.add(eleve_db)
    session.commit()
    session.refresh(eleve_db) # Récupère l'id auto-généré par SQL
```

3. ANTISÈCHE CRUD SQLMODEL / SQLALCHEMY 2.0

```
from sqlmodel import Session, select, func

with Session(engine) as session:
    # LIRE TOUT ou FILTRER
    stmt = select(Apprenant).where(Apprenant.is_active == True)
    actifs = session.exec(stmt).all()

    # LIRE PAR ID (Clé primaire)
    un_eleve = session.get(Apprenant, 1)

    # COMPTER (func.count)
    nb = session.exec(select(func.count(Apprenant.id))).one()

    # MODIFIER (Dirty Checking automatique)
    un_eleve.niveau_etude = "Bac+5"
    session.add(un_eleve)
    session.commit()

    # SUPPRIMER
    session.delete(un_eleve)
    session.commit()
```

1. DÉTECTER LES CONFLITS HORAIRES (SALLES, FORMATEURS, APPRENANTS)

📐 **Théorème d'intersection de deux intervalles temporels :**
Deux créneaux `[Debut_Existant, Fin_Existant]` et `[Debut_Nouveau, Fin_Nouveau]` se chevauchent si et seulement si :
`SessionFormation.date_debut < fin_nouvelle` **ET** `SessionFormation.date_fin > debut_nouvelle`.

Fichier : `app/utils/validators.py` (Salles & Formateurs)

```
from datetime import datetime
from sqlmodel import Session, select, func
from app.models.salles import Salle
from app.models.sessions import SessionFormation, StatutSessionEnum
from app.models.inscriptions import Inscription, StatutInscriptionEnum

def is_salle_disponible(
    session: Session, salle_id: int, debut: datetime, fin: datetime,
    exclude_session_id: int | None = None
) → bool:
    # Renvoie True si aucune session active n'occupe la salle sur ce créneau
    stmt = select(SessionFormation).where(
        SessionFormation.salle_id == salle_id,
        SessionFormation.statut ≠ StatutSessionEnum.ARCHIVED,
        SessionFormation.date_debut < fin,    # Commence avant la fin du nouveau
        SessionFormation.date_fin > debut     # Finit après le début du nouveau
    )
    if exclude_session_id:
        stmt = stmt.where(SessionFormation.id ≠ exclude_session_id)
    conflit = session.exec(stmt).first()
    return conflit is None

def is_formateur_disponible(
    session: Session, formateur_id: int, debut: datetime, fin: datetime
) → bool:
    # Vérifie que le formateur n'anime pas déjà une autre session en même temps
    stmt = select(SessionFormation).where(
        SessionFormation.formateur_id == formateur_id,
        SessionFormation.statut ≠ StatutSessionEnum.ARCHIVED,
        SessionFormation.date_debut < fin,
        SessionFormation.date_fin > debut
    )
    return session.exec(stmt).first() is None
```

Validation complète d'une Session & d'une Inscription

```
def valider_creation_session(session: Session, data: SessionCreate):
    # Vérifie la capacité de la salle + disponibilité salle et formateur
    salle = session.get(Salle, data.salle_id)
    if not salle or not salle.is_active:
        raise ValueError("Salle introuvable ou inactive.")
    if data.capacite_max > salle.capacite:
        raise ValueError(f"Capacité max ({data.capacite_max}) > salle
({salle.capacite}).")
    if not is_salle_disponible(session, data.salle_id, data.date_debut,
data.date_fin):
        raise ValueError("Conflit : Cette salle est déjà réservée sur ce créneau
!")
    if not is_formateur_disponible(session, data.formateur_id, data.date_debut,
data.date_fin):
        raise ValueError("Conflit : Ce formateur donne déjà cours sur ce créneau
!")

def valider_inscription_apprenant(session: Session, apprenant_id: int, session_id:
int):
    # Vérifie : 1. Session OPEN, 2. Places restantes, 3. Pas de conflit horaire
    cours = session.get(SessionFormation, session_id)
    if not cours or cours.statut ≠ StatutSessionEnum.OPEN:
        raise ValueError("Cette session n'est pas ouverte aux inscriptions.")

    nb_inscrits = session.exec(
        select(func.count(Inscription.id)).where(
            Inscription.session_id == session_id,
            Inscription.statut_inscription == StatutInscriptionEnum.ENREGISTRE
        )
    ).one()
    if nb_inscrits ≥ cours.capacite_max:
        raise ValueError("Session complète : capacité maximale atteinte !")

    stmt_conflit = (
        select(SessionFormation)
        .join(Inscription, Inscription.session_id == SessionFormation.id)
        .where(
            Inscription.apprenant_id == apprenant_id,
            Inscription.statut_inscription == StatutInscriptionEnum.ENREGISTRE,
            SessionFormation.date_debut < cours.date_fin,
            SessionFormation.date_fin > cours.date_debut
        )
    )
    if session.exec(stmt_conflit).first() is not None:
        raise ValueError("Conflit horaire : l'apprenant a déjà un cours sur ce
créneau !")
```

2. LE PIPELINE DE NETTOYAGE DES 2 000 UTILISATEURS (MIGRATION ERP → SQLMODEL)

Voici comment lire la base brute `legacy_erp_cftech.db` (qui contient ~13,5 % d'erreurs), filtrer automatiquement avec vos schémas Pydantic, et insérer uniquement les données propres dans votre nouvelle base `cftech.db` :

```
import sqlite3, json
from pydantic import ValidationError
from sqlmodel import Session
from app.database import engine, create_db_and_tables
from app.models.users import Apprenant, Formateur, StaffPedagogique, Admin
from app.schemas.user_schemas import ApprenantCreate, FormateurCreate, StaffCreate, AdminCreate

def migrer_et_nettoyer_legacy():
    create_db_and_tables()
    conn_legacy = sqlite3.connect("legacy_erp_cftech.db")
    conn_legacy.row_factory = sqlite3.Row
    lignes_brutes = conn_legacy.execute("SELECT * FROM legacy_erp_users").fetchall()

    valides, rejets, emails_vus = 0, [], set()

    with Session(engine) as session:
        for row in lignes_brutes:
            d = dict(row)
            try:
                # 1. Filtre anti-doublon d'email
                if not d["email"] or d["email"] in emails_vus:
                    raise ValueError(f"Email doublon ou vide : {d['email']}")

                # 2. Passage au crible Pydantic selon le rôle
                if d["role"] == "apprenant":
                    schema_propre = ApprenantCreate.model_validate(d)
                    obj_db = Apprenant(**schema_propre.model_dump())
                elif d["role"] == "formateur":
                    schema_propre = FormateurCreate.model_validate(d)
                    obj_db = Formateur(**schema_propre.model_dump())
                elif d["role"] == "staff":
                    if isinstance(d.get("responsabilites"), str):
                        d["responsabilites"] = json.loads(d["responsabilites"]) # Test JSON valide
                    schema_propre = StaffCreate.model_validate(d)
                    obj_db = StaffPedagogique(**schema_propre.model_dump())
                elif d["role"] == "admin":
                    schema_propre = AdminCreate.model_validate(d)
                    obj_db = Admin(**schema_propre.model_dump())
                else:
                    raise ValueError(f"Rôle inconnu dans l'ancien ERP : {d['role']}")

                emails_vus.add(d["email"])
                session.add(obj_db)
                valides += 1

            except (ValidationError, ValueError, json.JSONDecodeError) as err:
                # Les ~13.5% de lignes corrompues atterrissent ici sans faire planter le programme !
                rejets.append({"id_legacy": d["id"], "nom": d["nom"], "role": d["role"], "motif": str(err)})

        session.commit()
    print(f"✅ Migration terminée : {valides} insérés | ❌ {len(rejets)} rejetés ({len(rejets)/len(lignes_brutes)*100:.1f}%)")
    return valides, rejets
```

⏱️ Guide Pas-à-Pas Alembic & Architecture Finale

Le Grand Absent des Notebooks : Réussir les 4 Migrations Obligatoires sans Erreur

1. CONFIGURER ALEMBIC AVEC SQLMODEL (EN 3 MINUTES)

Par défaut, Alembic ne connaît pas `SQLModel`. Après avoir lancé `alembic init alembic`, vous devez modifier **2 fichiers** :

A. Dans `alembic/script.py.mako`

Ajoutez l'import `import sqlmodel` en haut du template pour que tous vos fichiers de migration reconnaissent le type `sqlmodel.sql.sqltypes.AutoString()` :

```
"""${message}"""
from typing import Sequence, Union
from alembic import op
import sqlalchemy as sa
import sqlmodel  # ⟵ AJOUTER CETTE LIGNE OBLIGATOIREMENT !
${imports if imports else ""}
```

B. Dans `alembic/env.py`

Branchez votre `DATABASE_URL` et `SQLModel.metadata` :

```
import os
from dotenv import load_dotenv
from sqlmodel import SQLModel
import app.models  # ⟵ Charge vos 7 tables !

load_dotenv()
config = context.config
# Injecte l'URL du fichier .env dans Alembic
config.set_main_option("sqlalchemy.url", os.getenv("DATABASE_URL",
"sqlite:///./cftech.db"))

# Indique à Alembic de surveiller vos tables SQLModel
target_metadata = SQLModel.metadata
```

💡 **Astuce spéciale SQLite dans `alembic/env.py` :** Dans la fonction `run_migrations_online()`, ajoutez `render_as_batch=True` dans `context.configure( ... , target_metadata=target_metadata, render_as_batch=True)`. Cela permet à SQLite de modifier/ajouter des colonnes sans jamais bloquer !

2. LES 4 MIGRATIONS OBLIGATOIRES DU BRIEF (CHRONOLOGIE EXACTE)

1️⃣ Migration v1 (Initiale)
7 tables créées SANS `statut` (Session) et SANS `equipements` (Salle)

→ `--autogenerate` →

2️⃣ Migration v2 (Statut)
Ajout de la colonne `statut` (OPEN, CLOSED, ARCHIVED) dans `SessionFormation`

→ `--autogenerate` →

3️⃣ Migration v3 (JSON Salle)
Refonte de `Salle` : ajout de la colonne JSON `equipements`

→ `--autogenerate` →

4️⃣ Migration v4 (Rôles)
Évolution des rôles : ajout de `SUPERADMIN` dans `Admin`

Les Commandes Terminal à Exécuter à Chaque Étape

```
# Étape 1 : Commentez 'statut' (dans sessions.py) et 'equipements' (dans salles.py)
alembic revision --autogenerate -m "v1_creation_tables_initiales"
alembic upgrade head

# Étape 2 : Décommentez 'statut' dans app/models/sessions.py
alembic revision --autogenerate -m "v2_ajout_statut_session"
alembic upgrade head

# Étape 3 : Décommentez 'equipements' (JSON) dans app/models/salles.py
alembic revision --autogenerate -m "v3_refonte_salle_equipements_json"
alembic upgrade head

# Étape 4 : Ajoutez SUPERADMIN dans NiveauAccesEnum + commentaire explicite
alembic revision --autogenerate -m "v4_evolution_role_superadmin"
alembic upgrade head

# Vérifier que l'historique des 4 versions est propre pour la soutenance :
alembic history --verbose
```

Fonction Métier Demandée : `check_permission`

Le brief (Section 5) évalue explicitement la présence de `check_permission` dans `app/utils/permissions.py` :

```
# Fichier : app/utils/permissions.py
from app.models.users import UserBase

PERMISSIONS_PAR_ROLE = {
    "apprenant": {"voir_sessions", "s_inscrire"},
    "formateur": {"voir_sessions", "gerer_presences"},
    "staff": {"voir_sessions", "creer_session", "gerer_salles",
"gerer_inscriptions"},
    "admin": {"voir_sessions", "creer_session", "gerer_salles", "gerer_users"},
    "superadmin": {"ALL"}
}

def check_permission(user: UserBase, action: str) → bool:
    # Vérifie si un utilisateur actif possède le droit d'exécuter 'action'
    if not user or not user.is_active:
        return False
    if getattr(user, "niveau_acces", None) == "SUPERADMIN":
        return True
    droits = PERMISSIONS_PAR_ROLE.get(user.role, set())
    return "ALL" in droits or action in droits
```

3. TABLEAU RÉCAPITULATIF DE L'ARCHITECTURE DU DÉPÔT

Fichier du Projet	Outil / Cours	Contenu Exact & Responsabilité
`app/database.py`	Cours 008 & 009	Charge `.env`, crée `engine = create_engine( ... )`, fournit `get_session()` et `create_db_and_tables()`.
`app/models/users.py`	Cours 009 (SQLModel)	`UserBase(table=False)` avec `@property def role` + 4 sous-classes `table=True` (`Apprenant`, `Formateur`, `StaffPedagogique`, `Admin`).
`app/models/salles.py` `sessions.py`, `inscriptions.py`	Cours 009 (SQLModel)	Tables `Salle` (JSON `equipements`), `SessionFormation` (FK salle & formateur), `Inscription` (`UniqueConstraint` apprenant+session).
`app/schemas/*.py`	Cours 007 (Pydantic)	Classes `*Create` avec `EmailStr`, `@field_validator` (âge ≥ 16, regex tél, date embauche < today) et `@model_validator` (dates session).
`app/utils/validators.py` `app/utils/permissions.py`	Cours 008 (Requêtes)	Fonctions évaluées par le jury : `is_salle_disponible()`, `is_formateur_disponible()`, `valider_inscription_apprenant()`, `check_permission()`.
`alembic/versions/*.py`	Brief Section 3	Les 4 scripts de migration générés chronologiquement (v1 tables, v2 statut, v3 JSON salle, v4 superadmin).